MakeCode 시뮬레이터 LED 이름 표시 — 이론해설서

Changmo Yang & Claude AI

2026-02-19

시뮬레이터 버전: v1.0.2.4 | 문서 버전: v1.0.0.0

목차

1 개요 **3**
1.1 이 문서의 목적 3
1.2 다루는 주제 3

2 블록 코딩의 원리 **3**
2.1 블록 코딩이란? 3
2.2 블록 = 명령어 3
2.3 블록 코딩 vs 텍스트 코딩 4

3 이벤트 구동 프로그래밍 **4**
3.1 이벤트란? 4
3.2 이벤트 구동 모델 5
3.3 왜 이벤트 구동인가? 5

4 순차 실행 **6**
4.1 순차 실행이란? 6
4.2 순서가 중요한 이유 6
4.3 프로그래밍의 3대 제어 구조 6

5 반복 구조 **7**
5.1 반복이란? 7
5.2 무한 루프의 필요성 7
5.3 무한 루프를 멈추는 방법 7

6 LED 매트릭스와 비트맵 폰트 **8**
6.1 LED 매트릭스란? 8
6.2 비트맵 폰트 9
6.3 5×5의 한계와 공학적 타협 9
6.4 디지털 화면의 원리 9

7 시뮬레이터의 기술적 원리 **10**
7.1 시뮬레이터란? 10
7.2 드래그 앤 드롭의 원리 10
7.3 비동기 실행의 원리 10

8 핵심 개념 요약 **10**

9 더 알아보기 **12**

9.1 관련 학습 자료 12
9.2 생각해보기 12

참고자료 **13**

생성/수정 이력 **13**

1. 개요

1.1 이 문서의 목적

본 이론해설서는 MakeCode 시뮬레이터 뒤에 숨겨진 컴퓨터 과학 개념과 프로그래밍 원리를 설명한다. 시뮬레이터를 "사용"하는 것을 넘어 "왜 그렇게 동작하는지"를 이해하는 것이 목표이다. 별도 학습 시간에 읽거나, 수업 후 심화 자료로 활용한다.

1.2 다루는 주제

주제	핵심 질문
블록 코딩	왜 블록 코딩이 존재하는가? 텍스트 코딩과 무엇이 다른가?
이벤트 구동 프로그래밍	"시작하면 실행"은 어떤 프로그래밍 패러다임인가?
순차 실행	컴퓨터는 왜 위에서 아래로 한 줄씩 실행하는가?
반복 구조	"무한반복"은 왜 필요하고, 어떻게 멈추는가?
LED 매트릭스	5×5 점으로 어떻게 글자를 표현하는가?
비트맵 폰트	디지털 화면에서 문자는 어떻게 만들어지는가?

2. 블록 코딩의 원리

2.1 블록 코딩이란?

블록 코딩(Block-based Programming)은 텍스트 대신 시각적 블록을 조합하여 프로그램을 작성하는 방식이다. MIT의 스크래치(Scratch), micro:bit의 MakeCode, 엔트리(Entry) 등이 대표적이다.

핵심 아이디어: 프로그래밍의 본질(논리적 사고)에 집중하고, 비본질적 장벽(문법 오류, 오타, 세미콜론)을 제거한다.

2.2 블록 = 명령어

시뮬레이터의 각 블록은 하나의 **명령어(Instruction)**에 대응한다. "문자열 보여주기 Hello!"라는 블록은 컴퓨터에게 "LED에 Hello!를 표시하라"는 지시를 전달한다.

블록 (시각적)	텍스트 코드 (MakeCode JavaScript)	의미
문자열 보여주기 `Hello!`	`basic.showString("Hello!")`	문자열을 LED에 표시
숫자 보여주기 `0`	`basic.showNumber(0)`	숫자를 LED에 표시
아이콘 보여주기 ❤️	`basic.showIcon(IconNames.Heart)`	아이콘을 LED에 표시
일시정지 `1000` ms	`basic.pause(1000)`	1000밀리초 대기
화면 지우기	`basic.clearScreen()`	LED 전체 끄기

블록 코딩은 텍스트 코딩의 "번역"이 아니라 같은 논리를 다른 표현 방식으로 나타낸 것이다. 블록을 드래그하여 조합하는 행위 자체가 프로그래밍이다.

2.3 블록 코딩 vs 텍스트 코딩

비교 항목	블록 코딩	텍스트 코딩
문법 오류 가능성	거의 없음 (블록이 맞지 않으면 끼울 수 없음)	높음 (오타, 괄호 누락 등)
진입 장벽	낮음	높음
표현력	제한적 (블록 종류에 의존)	무제한
실행 속도	느림 (해석 계층 추가)	빠름
대규모 프로젝트	비효율적 (블록이 많아지면 복잡)	효율적 (파일 분리, 함수 등)
학습 순서	초급 → 블록	중급 이후 → 텍스트

> ★ **교육적 시사점**: 블록 코딩은 프로그래밍의 "최종 목표"가 아니라 "학습 도구"이다. 블록으로 논리적 사고를 익힌 뒤, 텍스트 코딩으로 전환하는 것이 일반적인 교육 경로이다.

---

3. 이벤트 구동 프로그래밍

3.1 이벤트란?

이벤트(Event)는 프로그램 외부에서 발생하는 "사건"이다. 버튼 클릭, 타이머 만료, 센서 감지 등이 모두 이벤트에 해당한다.

시뮬레이터에서 다루는 두 가지 이벤트:

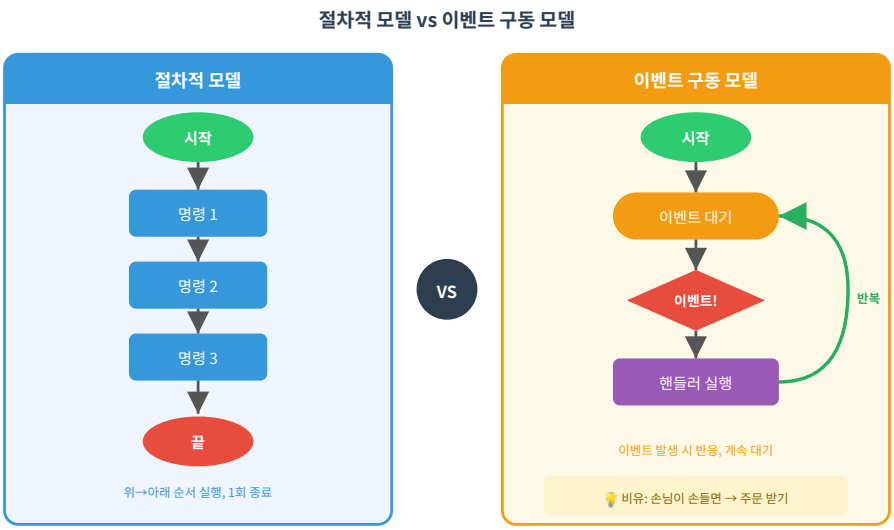

그림 1: 이벤트 구동 모델

이벤트	발생 시점	실행 방식
시작하면 실행 (on start)	프로그램이 처음 시작될 때	1회 실행 후 종료
무한반복 실행 (forever)	프로그램이 실행되는 동안	끝없이 반복

3.2 이벤트 구동 모델

전통적인 프로그래밍은 "위에서 아래로 순서대로 실행"하는 **절차적 모델**이다. 이벤트 구동 프로그래밍(Event-Driven Programming)은 "이벤트가 발생하면 지정된 동작을 실행"하는 **반응형 모델**이다.

```
절차적
[ 모델]시작
 → 명령1 → 명령2 → 명령3 → 끝이벤트

[ 구동 모델]시작
 → 이벤트 대기 → 이벤트( 발생!) → 핸들러 실행 → 이벤트 대기 → ...
```

micro:bit의 MakeCode는 두 모델을 **혼합**한다. `on start`는 절차적으로 한 번 실행되고, 이후 `forever`와 버튼 이벤트가 반응형으로 작동한다.

3.3 왜 이벤트 구동인가?

물리적 세계의 장치(IoT 기기, 센서, 버튼)는 **언제 입력이 들어올지 예측할 수 없다**. 사용자가 A 버튼을 1초 뒤에 누를 수도, 10분 뒤에 누를 수도 있다. 이벤트 구동 모델은 이런 불확실성에 자연스럽게 대응한다.

> ★ **일상 비유**: 음식점 주문 시스템을 생각해 보자. 종업원은 "손님이 손을 들면(이벤트) 주문을 받으러 간다(핸들러)". 모든 테이블을 1번부터 순서대로 방문하는 것(절차적)보다 효율적이다.

그림 2: 순차 실행 블록

4. 순차 실행

4.1 순차 실행이란?

순차 실행(Sequential Execution)은 프로그램의 명령어를 **작성된 순서대로** 하나씩 실행하는 것이다. 이것은 컴퓨터 프로그래밍의 가장 기본적인 원칙이다.

시뮬레이터에서 "시작하면 실행" 영역에 다음과 같이 블록을 배치하면:

반드시 1 → 2 → 3 순서로 실행된다. "Hi!" 표시 → 1초 대기 → ❤ 표시. 순서를 바꾸면 결과도 바뀐다.

4.2 순서가 중요한 이유

같은 블록 3개라도 순서에 따라 전혀 다른 결과가 나온다.

순서 A	순서 B
1. 문자열 `Hi!`	1. 아이콘 ❤
2. 일시정지 1000ms	2. 일시정지 1000ms
3. 아이콘 ❤	3. 문자열 `Hi!`
결과: Hi! → (1초) → ❤	**결과**: ❤ → (1초) → Hi!

> ★ **일상 비유**: 요리 레시피에서 "물을 끓인다 → 면을 넣는다 → 5분 기다린다"의 순서를 "면을 넣는다 → 5분 기다린다 → 물을 끓인다"로 바꾸면 결과가 완전히 달라진다.

4.3 프로그래밍의 3대 제어 구조

컴퓨터 프로그래밍은 3가지 기본 제어 구조의 조합으로 모든 알고리즘을 표현할 수 있다.

제어 구조	의미	시뮬레이터 대응
순차 (Sequence)	위에서 아래로 순서대로	블록 배치 순서
반복 (Iteration)	같은 동작을 여러 번	"무한반복 실행"
선택 (Selection)	조건에 따라 다른 동작	(본 시뮬레이터 미지원)

본 시뮬레이터는 순차와 반복을 다루며, 선택(if-else)은 다루지 않는다. 선택 구조는 향후 센서 데이터를 활용하는 2주차 이후에 학습한다.

---

5. 반복 구조

5.1 반복이란?

반복(Loop/Iteration)은 같은 명령을 여러 번 실행하는 것이다. "무한반복 실행"은 프로그램이 종료될 때까지(정지 버튼을 누를 때까지) 영원히 반복하는 **무한 루프(Infinite Loop)**이다.

5.2 무한 루프의 필요성

micro:bit 같은 임베디드 장치는 한 번 켜지면 **계속 동작**해야 한다. 온도를 1회만 측정하고 멈추는 온도계는 쓸모없다. 무한 루프가 있어야 센서 값을 지속적으로 읽고, LED를 계속 업데이트하고, 버튼 입력을 대기할 수 있다.

```
일상
[ 비유: 시계]시계는
 초침을" 초1 이동 → 표시 → 초침을 초1 이동 → 표시 → 를..."영원히
 반복한다. 이것이 무한 루프이다.
```

5.3 무한 루프를 멈추는 방법

방법	시뮬레이터	실제 micro:bit
사용자 중단	■ 정지 버튼	전원 끄기 (배터리 분리)
조건 중단	미지원	`break` 문 또는 조건부 탈출
외부 인터럽트	미지원	리셋 버튼

> ▲ **무한 루프와 "멈춤"**: 잘못 설계된 무한 루프(탈출 조건 없음)는 프로그램을 영원히 멈추지 못하게 하는 **버그**가 될 수 있다. 의도적인 무한 루프(forever)는 "언제든 멈출 수 있는" 장치가 반드시 함께 있어야 한다.

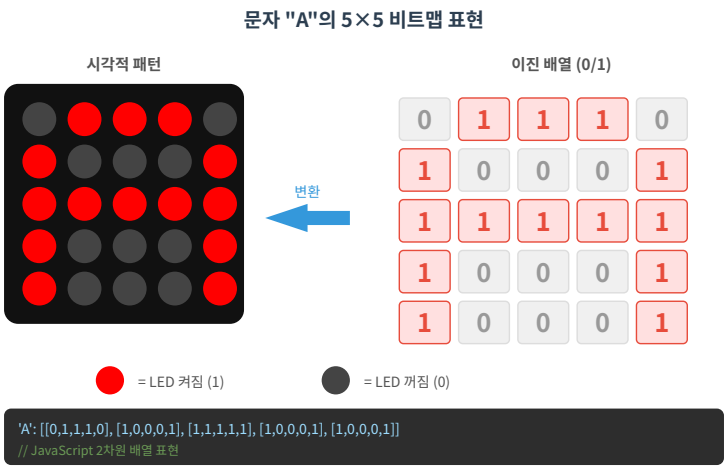

그림 3: LED 비트맵

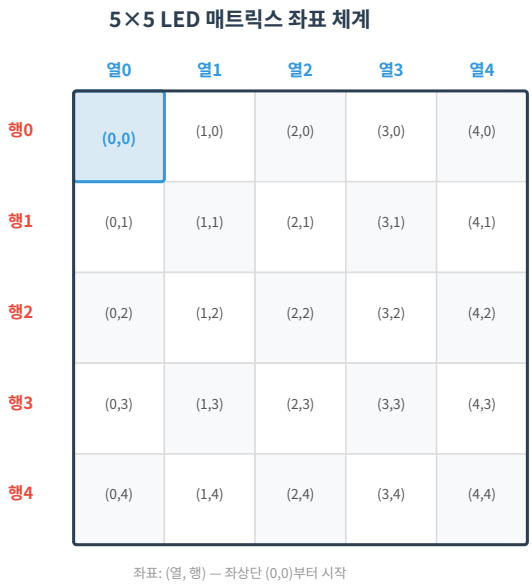

그림 4: LED 좌표 체계

6. LED 매트릭스와 비트맵 폰트

6.1 LED 매트릭스란?

LED 매트릭스(LED Matrix)는 LED를 격자(Grid) 형태로 배열한 디스플레이 장치이다. micro:bit는 5행 × 5열 = 25개 LED로 구성된 매트릭스를 내장한다.

```
열
:  0   1   2   3   4행
0 [ ] [ ] [ ] [ ] [ ]행
1 [ ] [ ] [ ] [ ] [ ]행
2 [ ] [ ] [ ] [ ] [ ]행
3 [ ] [ ] [ ] [ ] [ ]행
4 [ ] [ ] [ ] [ ] [ ]
```

각 LED는 **켜짐(1)** 또는 **꺼짐(0)**의 두 상태만 갖는다. 이 단순한 구조로 문자, 숫자, 아이콘을 표현한다.

6.2 비트맵 폰트

비트맵 폰트(Bitmap Font)는 문자를 **격자 위의 점(픽셀) 패턴**으로 정의한 것이다. 5×5 LED에서 글자 "A"는 다음과 같이 정의된다:

```
"A의" 5×5 비트맵:          이진 표현:
.  #  #  #  .             0 1 1 1 0
```

= LED 켜짐(1), . = LED 꺼짐(0)

시뮬레이터 코드에서는 이를 2차원 배열로 저장한다:

```
'A': [
  [0,1,1,1,0],
  [1,0,0,0,1],
  [1,1,1,1,1],
  [1,0,0,0,1],
  [1,0,0,0,1]
]
```

6.3 5×5의 한계와 공학적 타협

5×5 = 25개 점으로 문자를 표현하는 것은 **극도로 제한적**이다.

문자 유형	5×5 표현 가능성
영어 대문자 (A~Z)	✓ 대부분 가능
숫자 (0~9)	✓ 모두 가능
한글 자음 (ㄱ~ㅎ)	▲ 일부 유사하게
한글 완성형 (가~힣)	☒ 불가능
한자	☒ 불가능

이것은 **공학적 타협(Engineering Trade-off)**의 결과이다. LED 수를 늘리면 표현력이 올라가지만 비용, 전력 소모, 회로 복잡성, 크기가 함께 증가한다. micro:bit는 교육용 목적에서 "최소한의 하드웨어로 최대한의 학습 경험"을 추구하여 5×5를 선택했다.

> ★ **생각해보기**: 만약 LED가 8×8이라면 한글을 표현할 수 있을까? 16×16은? 실제로 한글을 전자 기기에 표현하려면 최소 몇 × 몇 격자가 필요할까?

6.4 디지털 화면의 원리

5×5 LED 매트릭스는 **모든 디지털 디스플레이**의 축소판이다. 스마트폰, 컴퓨터 모니터, TV 모두 작은 점(픽셀)의 격자로 이루어져 있다.

디스플레이	해상도	총 픽셀 수
micro:bit LED	5×5	25
옛날 게임보이	160×144	23,040
Full HD 모니터	1920×1080	2,073,600
4K TV	3840×2160	8,294,400

원리는 동일하다. 점을 **충분히 많이, 충분히 작게** 배열하면 사람의 눈에는 연속적인 이미지처럼 보인다. micro:bit의 5×5는 이 원리를 "가장 단순하게" 체험할 수 있는 교육 도구이다.

---

7. 시뮬레이터의 기술적 원리

7.1 시뮬레이터란?

시뮬레이터(Simulator)는 실제 장치의 동작을 소프트웨어로 **모방**하는 프로그램이다. 본 시뮬레이터는 실제 micro:bit의 LED 동작을 웹 브라우저 안에서 재현한다.

비교 항목	실제 micro:bit	시뮬레이터
LED	물리적 발광 다이오드	CSS로 색상 변경된 `<div>` 요소
프로그램 실행	ARM Cortex-M4 프로세서	JavaScript 엔진 (브라우저)
프로그래밍	MakeCode → .hex 파일	블록 드래그 → JavaScript 함수 호출
실행 시간	실시간 (마이크로초 단위)	근사적 (setTimeout 정밀도 한계)

7.2 드래그 앤 드롭의 원리

HTML5 Drag and Drop API는 다음 이벤트 시퀀스로 동작한다:

7.3 비동기 실행의 원리

시뮬레이터의 블록 실행 엔진은 JavaScript의 **async/await** 패턴을 사용한다. "일시정지 1000ms" 블록은 실제로 1초 동안 프로그램을 멈추는 것이 아니라, 1초 후에 다음 동작을 예약하는 것이다.

이 방식 덕분에 시뮬레이터가 실행 중에도 브라우저가 멈추지 않고, 정지 버튼이 즉시 반응할 수 있다.

---

8. 핵심 개념 요약

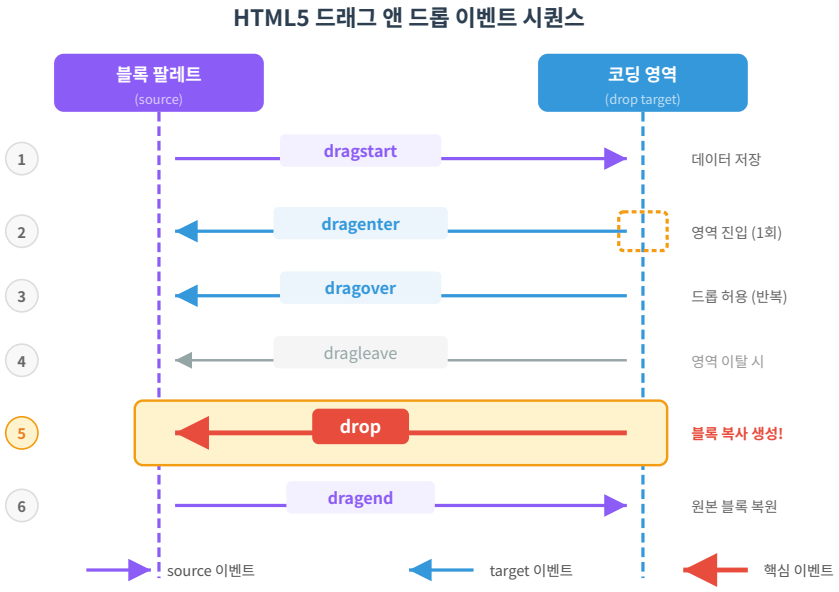

그림 5: DnD 시퀀스

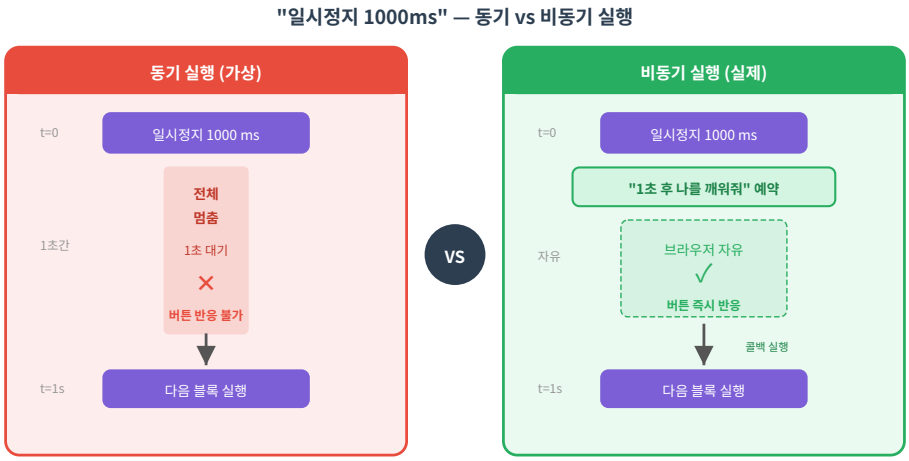

그림 6: 동기/비동기 비교

개념	학술 용어	시뮬레이터 대응	쉬운 설명
블록 코딩	Block-based Programming	팔레트의 블록	레고처럼 끼우는 코딩
명령어	Instruction	각 블록 1개	컴퓨터에게 내리는 지시
이벤트	Event	시작하면/무한반복	"~하면 ~해라"의 조건
핸들러	Event Handler	이벤트 안의 블록 묶음	이벤트에 반응하는 동작
순차 실행	Sequential Execution	블록 위→아래 순서	순서대로 하나씩
반복	Loop / Iteration	무한반복 실행	같은 일을 계속 반복
무한 루프	Infinite Loop	forever 영역	멈출 때까지 영원히
비트맵	Bitmap	5×5 LED 패턴 배열	점으로 그리는 그림
매트릭스	Matrix	5행×5열 LED 격자	격자 모양 점 배열
시뮬레이터	Simulator	이 프로그램 전체	진짜를 흉내 내는 프로그램
비동기	Asynchronous	일시정지 블록의 대기	"나중에 해줘" 예약

---

9. 더 알아보기

9.1 관련 학습 자료

주제	추천 자료
micro:bit 공식 튜토리얼	https://makecode.microbit.org/
블록 코딩 → 텍스트 코딩 전환	MakeCode의 JavaScript/Python 뷰 전환 기능
LED 매트릭스 심화	micro:bit LED 좌표 시스템 공식 문서
이벤트 구동 프로그래밍	스크래치의 "~를 클릭했을 때" 블록과 비교 학습

9.2 생각해보기

Q1: 블록 코딩이 텍스트 코딩보다 항상 쉬운가요? 블록이 수백 개가 되면 어떤 문제가 생길까요?

Q2: micro:bit의 5×5 LED 대신 1×1 LED(불 하나)만 있다면 어떤 정보를 전달할 수 있을까요? (힌트: 모스 부호)

Q3: "시작하면 실행" 영역에 블록을 1000개 넣으면 어떻게 될까요? 실행 시간은 어떻게 변할까요?

Q4: 실제 신호등도 "무한 루프"로 동작할까요? 신호등의 "이벤트"는 무엇일까요?

참고자료

#	자료명	유형	비고
1	사용설명서_v1.0.0.0	프로젝트 문서	전체 기능 상세
2	활용가이드_v1.0.0.0	프로젝트 문서	수업 적용 방법
3	Resnick, M. et al. (2009). Scratch: Programming for All	논문	블록 코딩 교육 연구
4	micro:bit Educational Foundation	웹사이트	https://microbit.org/teach/
5	코딩관리지침_v4.0.0.0	프로젝트 문서	문서화 작성법

프로젝트 문서의 정식 접두어는 “MakeCode시뮬레이터LED이름표시_”입니다.

생성/수정 이력

버전	날짜	시간	변경 수준	변경 내용	작성자
v1.0.0.0	2026-02-19	—	최초	초안 작성 (시뮬레이터 v1.0.2.4 기준)	Claude

MakeCode 시뮬레이터 LED 이름 표시 — 이론해설서